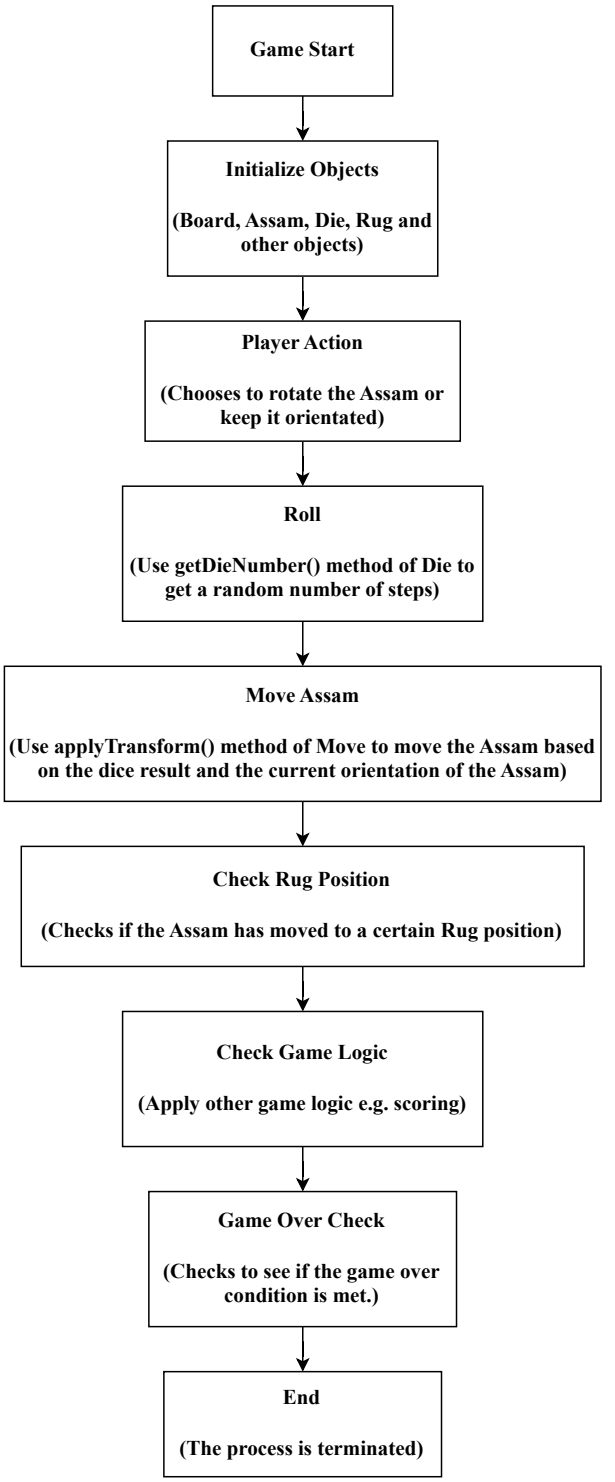
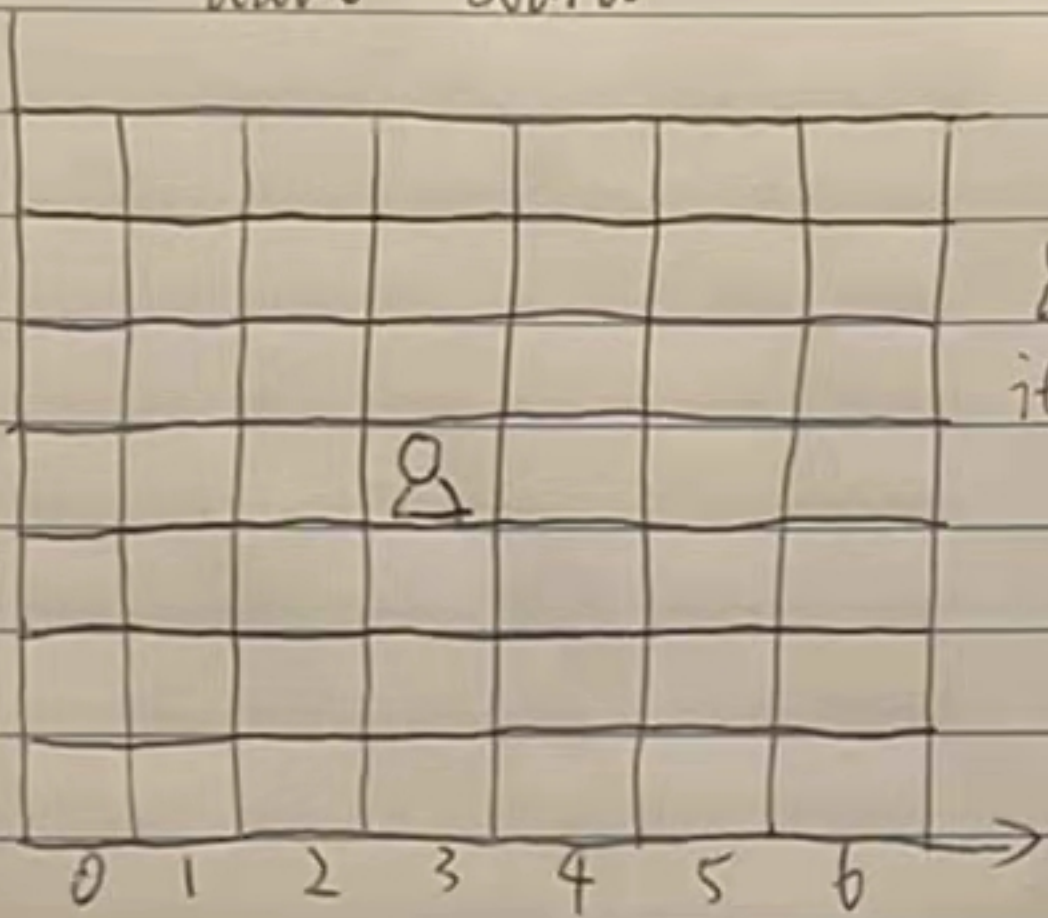
*Diagram: Flow Chart*



1. Game start about board




is the guest, it start at (3,3).

This is a 7×7 board, I consider ~~it's~~ the (0,0) in the bottom left corner, so that we can get:

1. (x,y) stand for the position $(0 \le x \le 6)(0 \le y \le 6)$
2. when (x+1), it means it move to right side OF THE BOARD (⚠ if the guest face down side and move right, then it should be (x-1), there will be many similar situation, same thing for y)
3. When the customer front close beyond the board, the customer will turn ~~left~~ (According to the costomer) left

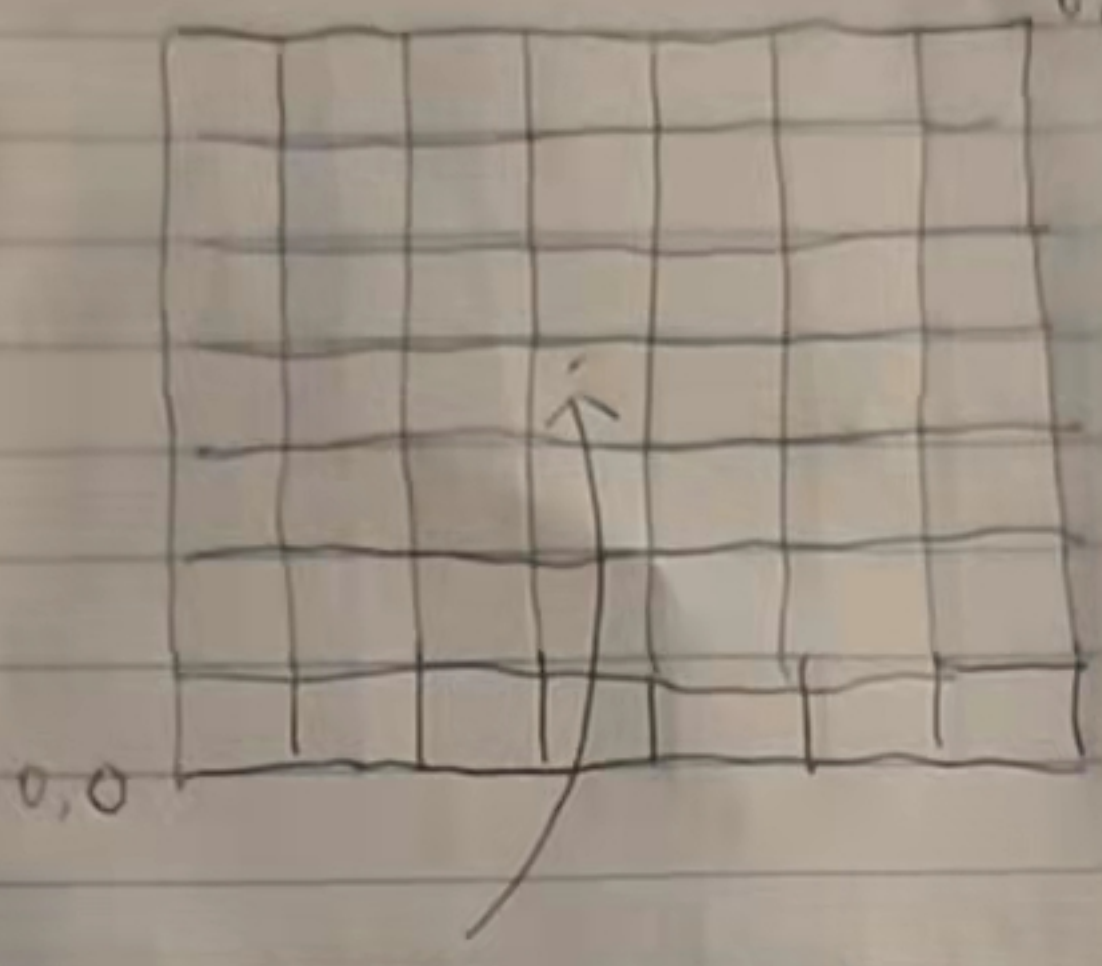
about player

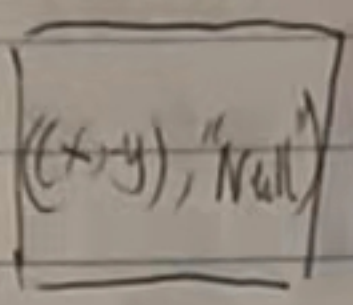
1. Colour: ① Blue ② Red ③ Yellow ④ Brown
At least two, four for most

2. Choose the number of players, two will be ①, ②,
three will be ①, ②, ③, four will be ① ② ③ ④.

## 2. Game Process

In the backend, the Board should by looks like:




(x,y) is the coordinate,
"Null" stand for the owner.

I have four lists to store every single players' places
e.g: list <intPair> redPosition ------
(This is in "Board" class.)

Then these lists will be converted to lists of lists of intpair
e.g. list<list<intPair>> redPiece -------
(This is in "Rug" class)
It's aims to make adjacent rugs into a whole piece of rug.

"Die" class will generate a random number for "Move" class. Then "Move" class will take it as an input and apply a transformation to Assam.
Then Assam will change his status according to the status he has right now. (which is recorded in "Assam" class)

check Assam's position, if he stand on someone's place, check which player own this space.

Then assume Assam stand on (x,y), we check List<List<IntPair>>, find out which List<IntPair> has (x,y) as an element.

Once we find it out, calculate the length of this List<IntPair>, the result is the money that need to be paid.

Then we check out the game is over or not.